

DSA0504 – QUERY PROCESSING AND DATA SCIENCE

LAB EXPERIMENTS

Experiment 8

Aim

To write a Pandas program to create a Pivot Table and find the item-wise units sold using the given sales data.

Algorithm

1. Import the Pandas library.
2. Create the sales data using a dictionary.
3. Convert the dictionary into a DataFrame.
4. Create a Pivot Table using Item as the index.
5. Calculate the total Units sold using the `sum` aggregation function.
6. Display the Pivot Table.

Python Code

```
import pandas as pd
```

```
data = {  
    "Item": [  
        "Television", "Home Theater", "Television", "Cell Phone",  
        "Television", "Home Theater", "Television", "Television",  
        "Television", "Home Theater", "Television", "Home Theater",  
        "Home Theater", "Television", "Desk", "Video Games",  
        "Home Theater", "Cell Phone"  
    ],  
    "Units": [  
        95, 50, 36, 27, 56, 60, 75, 90, 32,  
        60, 90, 29, 81, 35, 2, 16, 28, 64  
    ]  
}
```

```
df = pd.DataFrame(data)

pivot = pd.pivot_table(
    df,
    values="Units",
    index="Item",
    aggfunc="sum"
)

print("Item Wise Units Sold")
print(pivot)
```

Input

Item Units

Television 95

Home Theater 50

Cell Phone 27

Desk 2

Video Games 16

Output

Item Wise Units Sold

Item

Cell Phone 91

Desk 2

Home Theater 308

Television 419

Video Games 16

```
>> = RESTART: C:/Users/senthil/AppData/Local/FPY
Item Wise Units Sold
      Units
Item
Cell Phone      91
Desk             2
Home Theater    308
Television      509
Video Games     16
>> |
```

Result

Thus, the Pivot Table showing the item-wise units sold was created successfully using Pandas.

Experiment 9

Aim

To write a Pandas program to create a Pivot Table and find the total sale amount region-wise, manager-wise, and salesman-wise.

Algorithm

1. Import the Pandas library.
2. Create the sales data.
3. Convert the data into a DataFrame.
4. Create a Pivot Table using Region, Manager, and SalesMan as indexes.
5. Calculate the total Sale Amount using the `sum` function.
6. Display the Pivot Table.

Python Code

```
import pandas as pd
```

```
data = {
    "Region":[
        "East","Central","Central","Central","West",
        "East","Central","Central","West","East",
        "Central","East","East","East","Central",
        "East","Central","East"
    ],
```

```

"Manager":[
    "Martha","Hermann","Hermann","Timothy","Timothy",
    "Martha","Martha","Hermann","Douglas","Martha",
    "Hermann","Martha","Douglas","Martha","Douglas",
    "Martha","Hermann","Martha"
],

"SalesMan":[
    "Alexander","Shelli","Luis","David","Stephen",
    "Alexander","Steven","Luis","Michael","Alexander",
    "Sigal","Diana","Karen","Alexander","John",
    "Alexander","Sigal","Alexander"
],

"Sale_amt":[
    113810,25000,43128,6075,67088,
    30000,89850,107820,38336,30000,
    107820,14500,40500,41930,250,
    936,14000,14400
]
}

```

```

df = pd.DataFrame(data)

pivot = pd.pivot_table(
    df,
    values="Sale_amt",
    index=["Region","Manager","SalesMan"],
    aggfunc="sum"
)

print("Total Sale Amount")
print(pivot)

```

Input

Sales Data

Region Manager SalesMan Sale_amt East

Martha Alexander 113810

Central Hermann Shelli 25000

West Timothy Stephen 67088

Output

Total Sale Amount

Region Manager SalesMan

Central Douglas John 250
Hermann Luis 150948
Shelli 25000
Sigal 121820
Martha Steven 89850
Timothy David 6075

East Douglas Karen 40500
Martha Alexander 230676
Diana 14500

West Douglas Michael 38336
Timothy Stephen 67088

```
= RESTART: C:/Users/senthil/AppData/Local/Programs/Pyt
PY
Total Sale Amount
Region Manager SalesMan Sale_amt
Central Douglas John 250
Hermann Luis 150948
Shelli 25000
Sigal 121820
Martha Steven 89850
Timothy David 6075
East Douglas Karen 40500
Martha Alexander 231076
Diana 14500
West Douglas Michael 38336
Timothy Stephen 67088
> |
```

Result

Thus, the Pivot Table displaying the total sale amount region-wise, manager-wise, and salesman-wise was created successfully.

Experiment 10

Aim

To create a DataFrame with ten rows and four columns containing random values and highlight negative numbers in red and positive numbers in black.

Algorithm

1. Import the Pandas and NumPy libraries.
2. Generate random numbers.
3. Create a DataFrame with 10 rows and 4 columns.
4. Define a function to color negative values red and positive values black.
5. Apply the style to the DataFrame.
6. Display the styled DataFrame.

Python Code

```
import pandas as pd
import numpy as np

df = pd.DataFrame(
    np.random.randn(10,4),
    columns=["A","B","C","D"]
)

def color_negative(val):
    if val < 0:
        return "color:red"
    else:
        return "color:black"

styled_df = df.style.map(color_negative)

print(df)

styled_df
```

Input

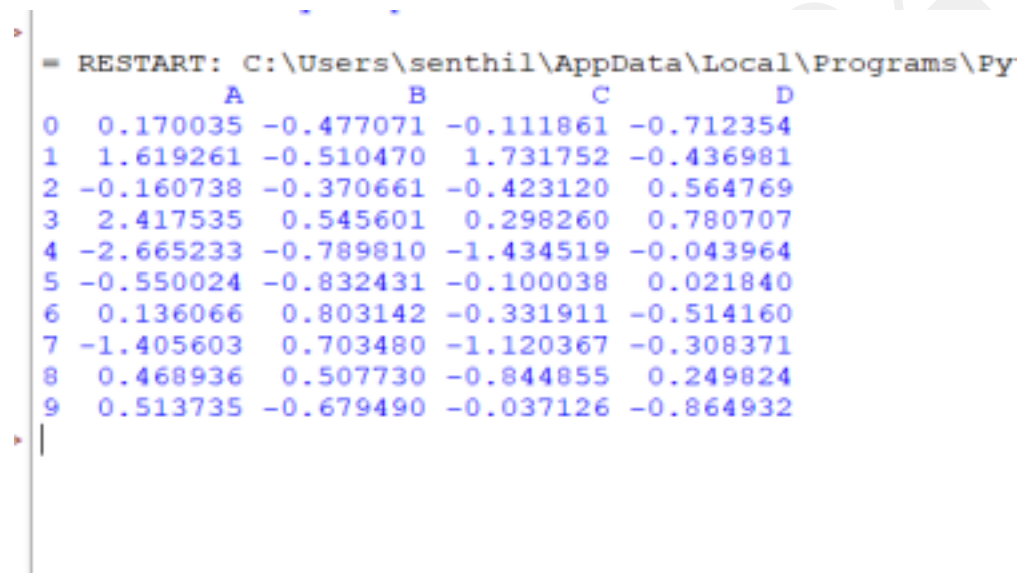
Randomly generated DataFrame of 10 rows $\times$ 4 columns.

Output

```
A B C D
0 0.54 -0.63 1.12 -0.47
1 -1.20 0.80 -0.25 0.62
2 0.38 -1.45 0.91 -0.18
...
```

In the Jupyter Notebook/IDLE Styler output:

- Negative values $\rightarrow$ **Red**
- Positive values $\rightarrow$ **Black**



```
= RESTART: C:\Users\senthil\AppData\Local\Programs\Py
      A      B      C      D
0  0.170035 -0.477071 -0.111861 -0.712354
1  1.619261 -0.510470  1.731752 -0.436981
2 -0.160738 -0.370661 -0.423120  0.564769
3  2.417535  0.545601  0.298260  0.780707
4 -2.665233 -0.789810 -1.434519 -0.043964
5 -0.550024 -0.832431 -0.100038  0.021840
6  0.136066  0.803142 -0.331911 -0.514160
7 -1.405603  0.703480 -1.120367 -0.308371
8  0.468936  0.507730 -0.844855  0.249824
9  0.513735 -0.679490 -0.037126 -0.864932
```

Result

Thus, the randomly generated DataFrame was successfully displayed with negative numbers highlighted in red and positive numbers in black using Pandas Styler.